WEEK 2 SUMMARY

Advanced C# and ASP.NET Core Foundations

Program: Backend Development Internship (.NET)

Week: Week 2 of 10 | Transition from Phase 1 to Phase 2

Theme: Advanced C#, concurrency, routing, middleware, and dependency injection

Duration: 40 hours across five training days

Executive Summary

Week 2 strengthened the C# foundation from Week 1 and introduced the infrastructure behind every ASP.NET Core API built later in the program. The work progressed from generics, LINQ, and asynchronous programming to a first Web API project with routing, middleware, and dependency injection.

The main outcome was a scaffolded and tested ASP.NET Core API, supported by reusable C# patterns. Interns learned how type safety, efficient querying, non-blocking concurrency, well-defined routes, correctly ordered middleware, and interface-based services work together to create maintainable backend applications.

Learning Objectives Achieved

- Created generic methods and classes with suitable type constraints.
- Used advanced LINQ operations for grouping, joining, flattening, and deliberate execution timing.
- Applied the Task-based asynchronous pattern without blocking on tasks.
- Scaffolded an ASP.NET Core Web API and implemented routes through Controllers and Minimal APIs.
- Explained middleware flow and configured pipeline order correctly.
- Registered and injected services using appropriate dependency-injection lifetimes.

Five-Day Curriculum and Practical Work

Day	Focus	Practical Outcome
1	Generics & Collections	Generic repository, constraints, and safe collection interfaces.
2	Advanced LINQ	Grouping, joining, SelectMany, deferred execution, and

Day	Focus	Practical Outcome
		performance awareness.
3	Async & Concurrency	Task-based async, Task.WhenAll, cancellation, and non-blocking patterns.
4	ASP.NET Core Routing	Scaffolded Web API, Controllers, Minimal APIs, and HTTP route semantics.
5	Middleware & DI	Pipeline order, custom middleware, service lifetimes, and constructor injection.

Day 1: Generics and Advanced Collections

Generics make reusable code type-safe. Instead of storing values as object and casting at runtime, types such as List<T> preserve compiler checks while supporting any appropriate type. Generic classes and methods declare type parameters, such as T or TEntity, that are resolved when the code is used.

Constraints such as where T : class, where T : IComparable, and where T : new() limit permitted type arguments and allow generic code to safely use required capabilities. The hands-on exercise implemented a generic Repository<T> with Add, GetAll, and predicate-based Find methods.

Collection design: Return the least permissive interface that satisfies the caller: IEnumerable<T> for forward iteration, IReadOnlyList<T> for count and indexing without mutation, and IList<T> only when callers must modify the collection. Avoid returning List<T> by default because it leaks mutability.

Day 2: Advanced LINQ and Deferred Execution

LINQ queries built with Where, Select, and OrderBy normally use deferred execution: they do not run until enumeration, such as foreach, ToList, ToArray, or Count. This is powerful but means results may change if the source data changes before the query is enumerated. Materialize with ToList at the point when a stable snapshot is needed.

- GroupBy clusters records by a shared key, for example total order value per customer.
- Join combines related collections on matching keys, similar to a SQL inner join.
- SelectMany flattens a nested collection into one sequence of inner items.
- Avoid unnecessary early ToList calls; however, materialize once and reuse the result when a deferred query would otherwise be enumerated repeatedly.

Day 3: Async/Await and Concurrency Basics

The Task-based asynchronous pattern allows work to continue without blocking a thread while I/O or another operation is in progress. Task represents a future operation, while Task<T> returns a result once

complete. Await resumes the method after completion without occupying the calling thread, enabling ASP.NET Core to serve many concurrent requests efficiently.

Async all the way: Do not use .Result or .Wait() in an async flow. They block threads and can cause deadlocks. Once a call chain becomes asynchronous, callers should also be async and use await consistently.

Independent operations should start together and be awaited with Task.WhenAll rather than awaited one after another. This reduces total latency from the sum of each task to roughly the duration of the slowest task. CancellationToken should be passed through long-running calls so work stops when a request is cancelled or times out.

Day 4: ASP.NET Core Web API Setup and Routing

The API project was scaffolded with dotnet new webapi, then run and explored through Swagger. Program.cs follows the minimal hosting model: WebApplicationBuilder configures services, WebApplication configures middleware, and Run starts the application.

Controllers organize related endpoints into classes and are the program default for larger APIs. Minimal APIs map endpoint lambdas directly in Program.cs and are useful for smaller services but can become difficult to organize at scale. Both approaches were implemented and tested through Postman.

Routing and verb semantics: Route templates such as /api/users/{id} bind URL segments to method parameters. GET reads without side effects, POST creates, PUT replaces, and DELETE removes. A GET endpoint must never change data because GET is expected to be safe to retry, cache, and prefetch.

Day 5: Middleware Pipeline and Dependency Injection

Every request passes through middleware in the order registered in Program.cs. Each middleware can inspect or modify the request, call the next component, and handle the response on its way back. This supports cross-cutting concerns such as logging, error handling, authentication, and authorization without duplicating code in each endpoint.

Middleware order is functional, not cosmetic. For example, authentication must run before authorization so the system knows who the user is before it checks permissions. Common components include HTTPS redirection, routing, authentication, authorization, and controller mapping.

- AddTransient creates a new service instance each time it is requested.
- AddScoped creates one instance per HTTP request and is the typical choice for request-bound services such as database contexts.
- AddSingleton creates one instance for the application's lifetime and must not hold request-specific mutable state.
- Constructor injection supplies dependencies through interfaces, allowing implementations to be replaced with test doubles later. Services should receive dependencies rather than constructing them internally.

Required Deliverables

- Generic Repository<T> with an appropriate constraint, committed to GitHub.
- LINQ exercises demonstrating GroupBy, Join, SelectMany, and deferred execution.
- Async concurrency demo using Task.WhenAll and a cancellation token.
- Scaffolded ASP.NET Core Web API with at least four endpoints using Controllers and Minimal APIs.
- Custom middleware plus an injected, DI-registered service.
- Week 2 summary prepared for the mentor check-in.

Testing and Review Checklist

1. Confirm generic repository callers cannot mutate a IReadOnlyList<T> result.
2. Demonstrate how deferred execution responds when the source collection changes before enumeration.
3. Compare sequential async calls with Task.WhenAll and verify cancellation behaves as intended.
4. Test Controller and Minimal API endpoints in Postman, including route-parameter behavior.
5. Verify custom middleware runs in the intended order and the injected service resolves with the correct lifetime.